Experiment 4B: CUDA Matrix Multiplication

This guide explains how to perform Matrix Multiplication using CUDA. The guide includes corrected CUDA code, installation steps, compilation instructions, sample input/output, and viva questions.

Problem Statement: Write a CUDA program for Matrix Multiplication.

Important: CUDA requires an NVIDIA GPU and CUDA Toolkit installation. This program will not run using AMD or Intel GPUs.

Correct CUDA Program Code

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define TILE_SIZE 16

// CUDA Kernel
__global__ void matrixMultiply(int *A, int *B, int *C, int N)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N)
    {
        int sum = 0;

        for (int k = 0; k < N; k++)
        {
            sum += A[row * N + k] * B[k * N + col];
        }

        C[row * N + col] = sum;
    }
}

int main()
{
    int N;

    printf("Enter size of square matrix (N x N): ");
    scanf("%d", &N);

    int size = N * N * sizeof(int);

    // Host memory
    int *h_A = (int*)malloc(size);
    int *h_B = (int*)malloc(size);
    int *h_C = (int*)malloc(size);

    // Input Matrix A
    printf("Enter elements of Matrix A:\n");

    for (int i = 0; i < N * N; i++)
    {
        scanf("%d", &h_A[i]);
    }

    // Input Matrix B
    printf("Enter elements of Matrix B:\n");

    for (int i = 0; i < N * N; i++)
    {
        scanf("%d", &h_B[i]);
    }

    // Device memory
```

```
    int *d_A, *d_B, *d_C;

    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);
    cudaMalloc((void**)&d_C, size);

    // Copy Host to Device
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // Thread configuration
    dim3 threadsPerBlock(TILE_SIZE, TILE_SIZE);

    dim3 blocksPerGrid(
        (N + TILE_SIZE - 1) / TILE_SIZE,
        (N + TILE_SIZE - 1) / TILE_SIZE
    );

    // Launch Kernel
    matrixMultiply<<<blocksPerGrid, threadsPerBlock>>>
    (d_A, d_B, d_C, N);

    cudaDeviceSynchronize();

    // Copy result back
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    // Display Result
    printf("\nResult Matrix C:\n");

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            printf("%d ", h_C[i * N + j]);
        }

        printf("\n");
    }

    // Free memory
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    free(h_A);
    free(h_B);
    free(h_C);

    return 0;
}
```

How to Run on Windows

1. Install NVIDIA CUDA Toolkit.
2. Open Command Prompt.
3. Save file as matrix_multiplication.cu.
4. Go to file location using cd command.
5. Compile using: nvcc matrix_multiplication.cu -o matrix_multiplication
6. Run using: matrix_multiplication

How to Run on Linux / Ubuntu

Compile and run:

```
sudo apt update

Install NVIDIA Driver
```

```
Install CUDA Toolkit

nvcc matrix_multiplication.cu -o matrix_multiplication

./matrix_multiplication
```

Sample Input

```
Enter size of square matrix (N x N): 2

Enter elements of Matrix A:
1 2
3 4

Enter elements of Matrix B:
5 6
7 8
```

Expected Output

```
Result Matrix C:
19 22
43 50
```

Matrix Multiplication Formula

```
C[i][j] = Σ(A[i][k] × B[k][j])
```

Important Viva Questions

Question	Answer
What is CUDA?	CUDA is NVIDIA's parallel computing platform.
Why use GPU for matrix multiplication?	Because many matrix operations run simultaneously.
What is a kernel?	A function executed on GPU.
What is TILE_SIZE?	Number of threads inside one block.
What does __global__ mean?	It defines a GPU kernel function.